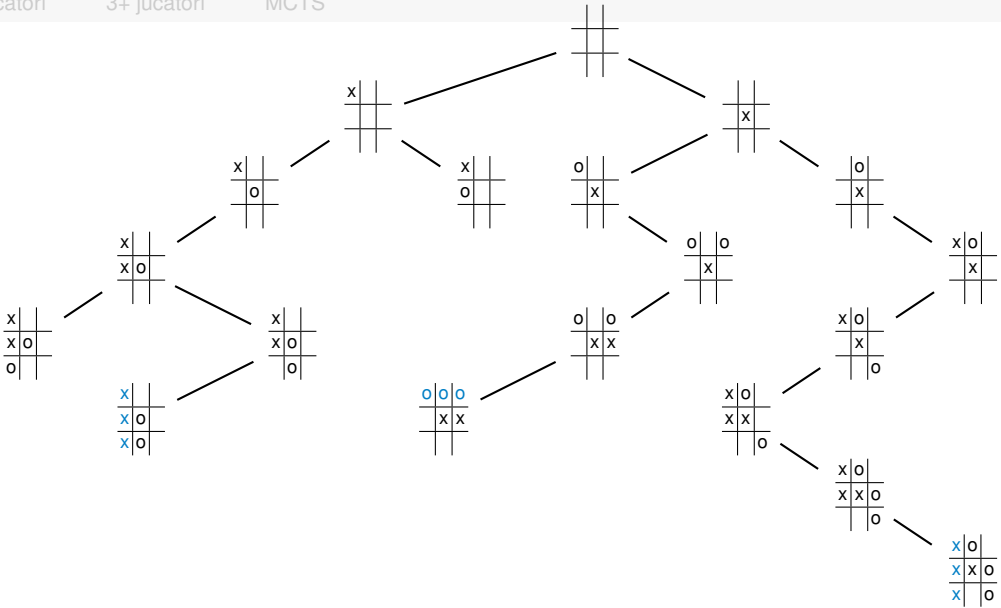
Inteligență Artificială

Căutare în jocuri

- vedem un joc ca o problemă de căutare – are un spațiu de stări, o stare inițială, și stări finale în care jocul se termină și în care jucătorii primesc o recompensă.



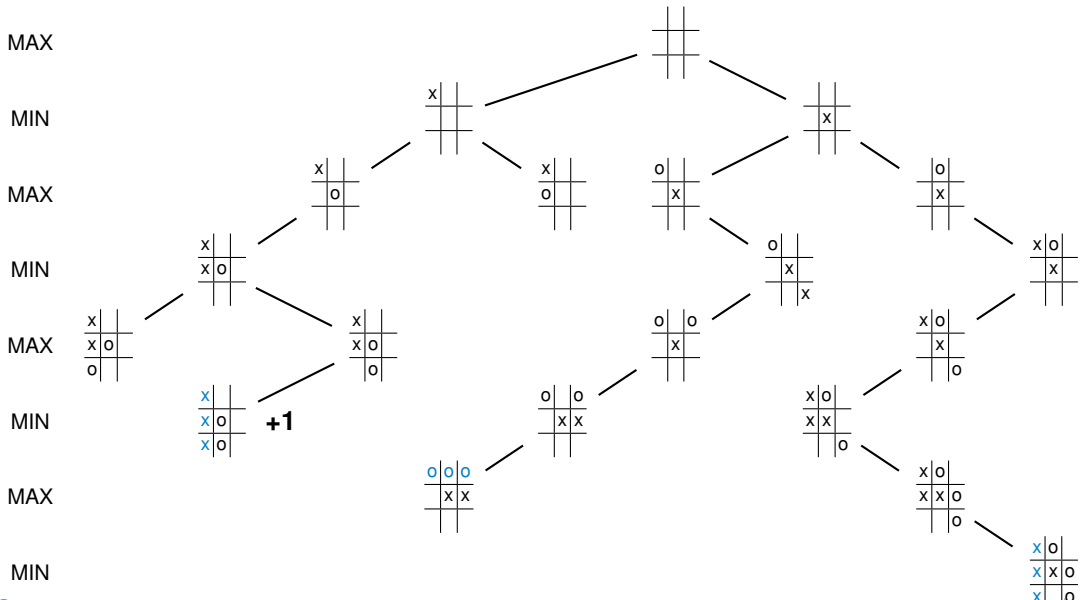
- jocuri
  - cooperative – Pandemic
  - non-cooperative – Poker
  - cu sumă 0 – X și 0, Șah, Go
- jocuri cu informație
  - perfectă – Șah, X și 0, Go
  - imperfectă – Poker, Bridge
  - cu elemente de șansă – Table, Poker
- jocuri
  - secvențiale – X și 0, Șah, Poker
  - cu mișcări simultane – Rock-paper-scissors

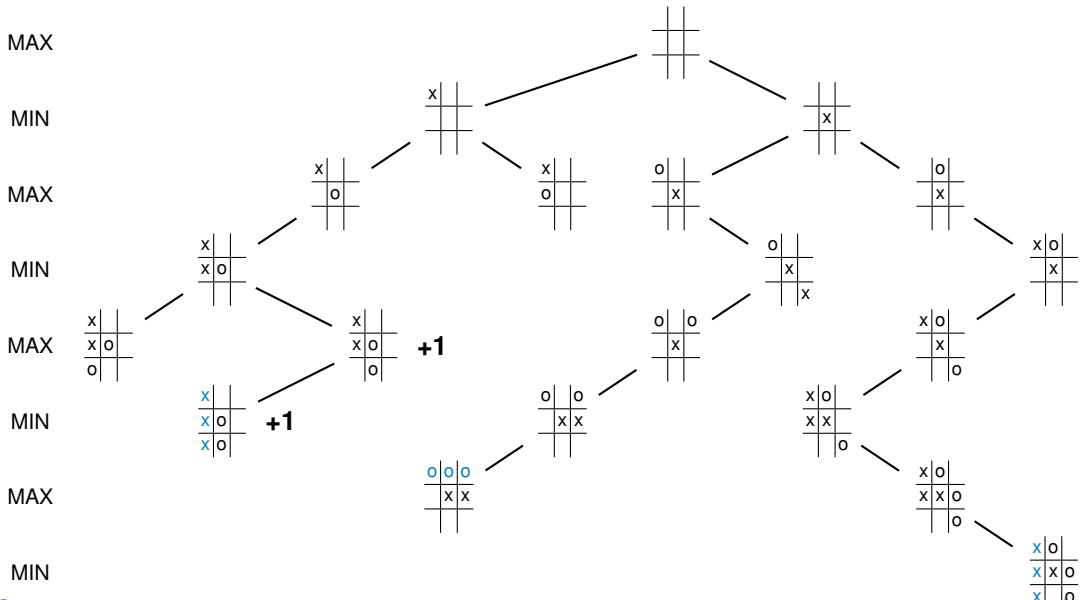
- ne referim mai ales la jocuri
  - cu sumă 0 – un jucător pierde și altul câștigă
  - cu informație perfectă – toate elementele sunt cunoscute ambilor jucători
  - secvențiale – jucătorii joacă pe rând

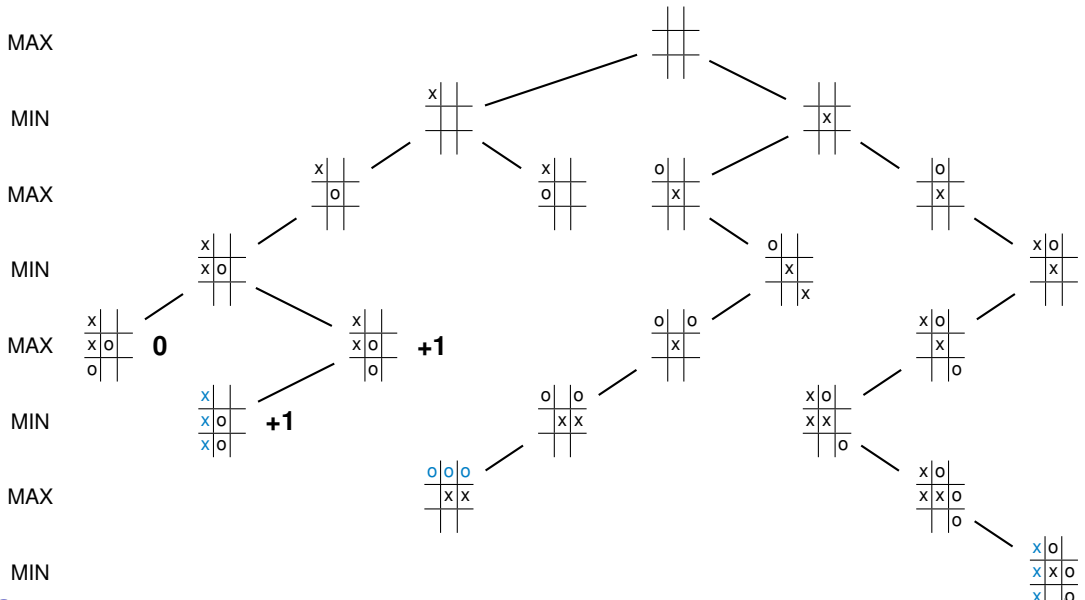
- pe bazele teoretice puse de von Neumann în 1928 și descris în 1944
- poate alege cea mai bună mutare pentru orice stare a jocului, dacă avem informație perfectă

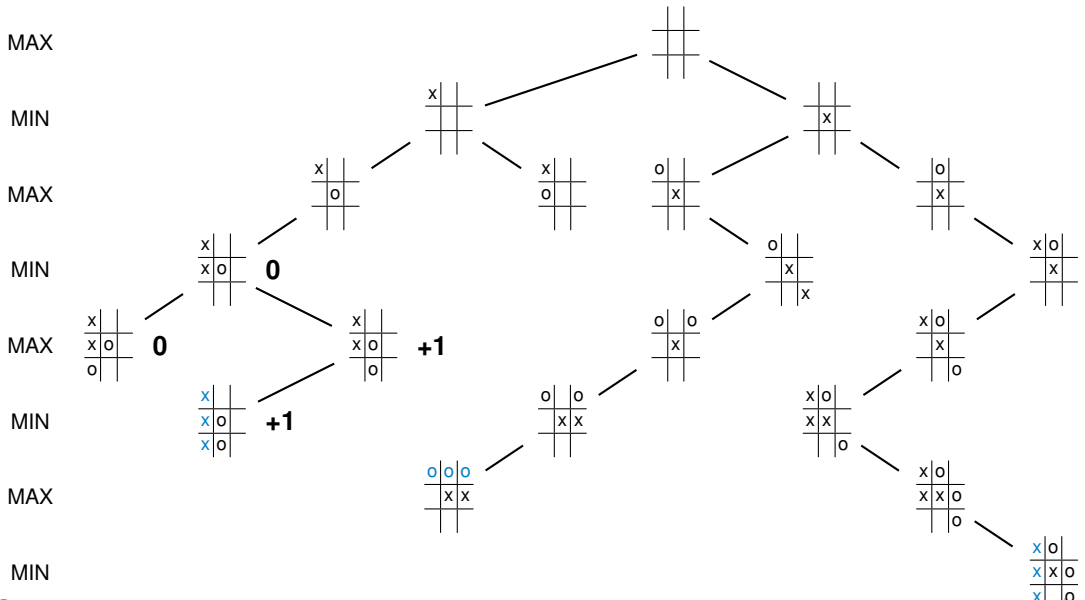
- jucătorul (“eu”) caută să își maximizeze câștigul
- adversarul caută să minimizeze câștigul jucătorului

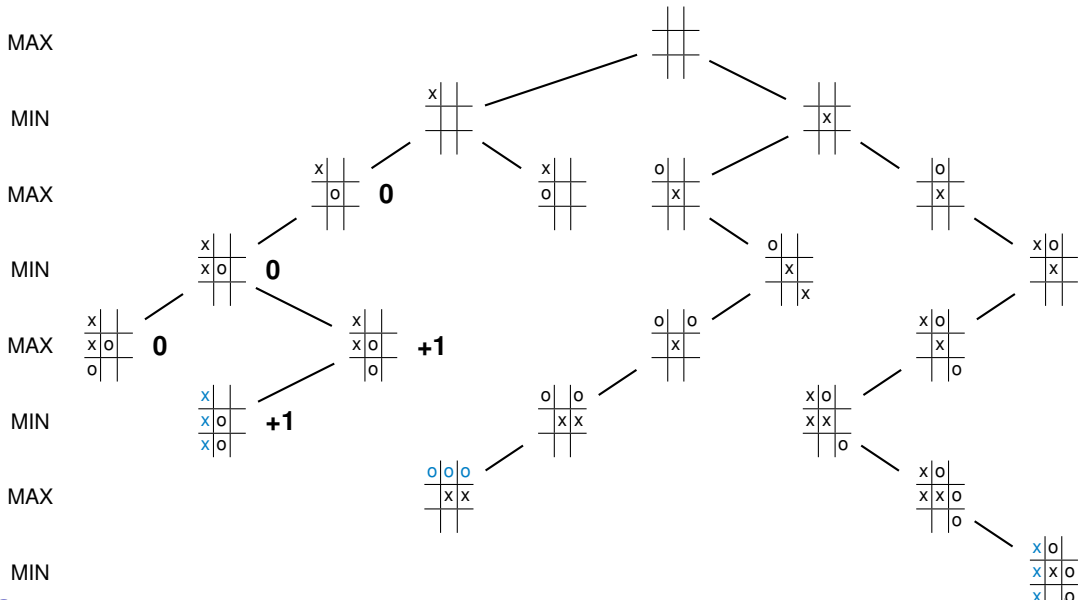
$\Rightarrow$ calculăm pentru fiecare nod din arborele de joc recompensa bazat pe cele 2 idei de mai sus.

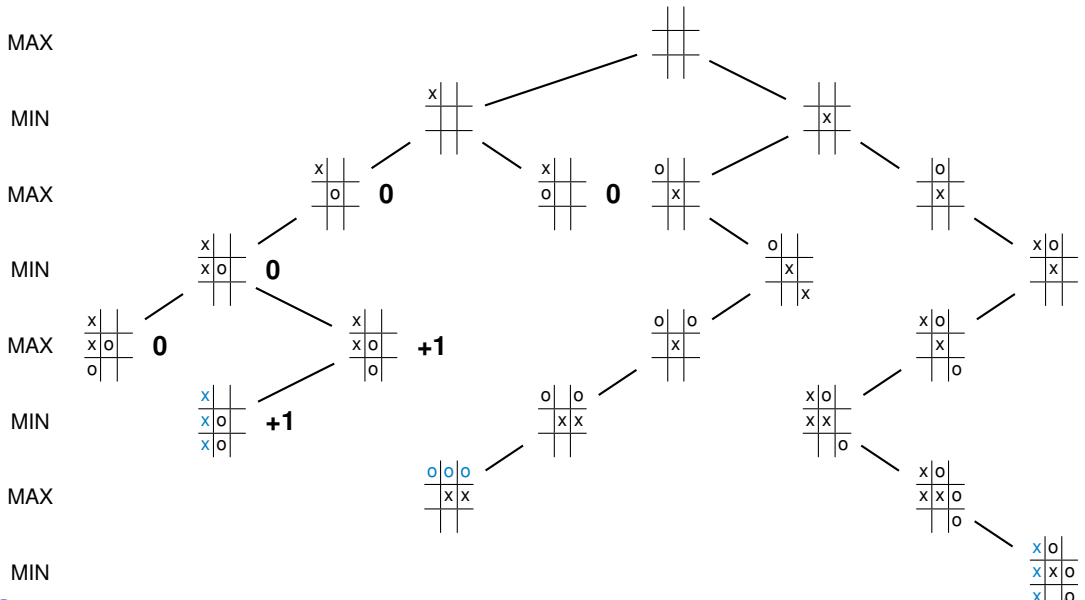

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
+1

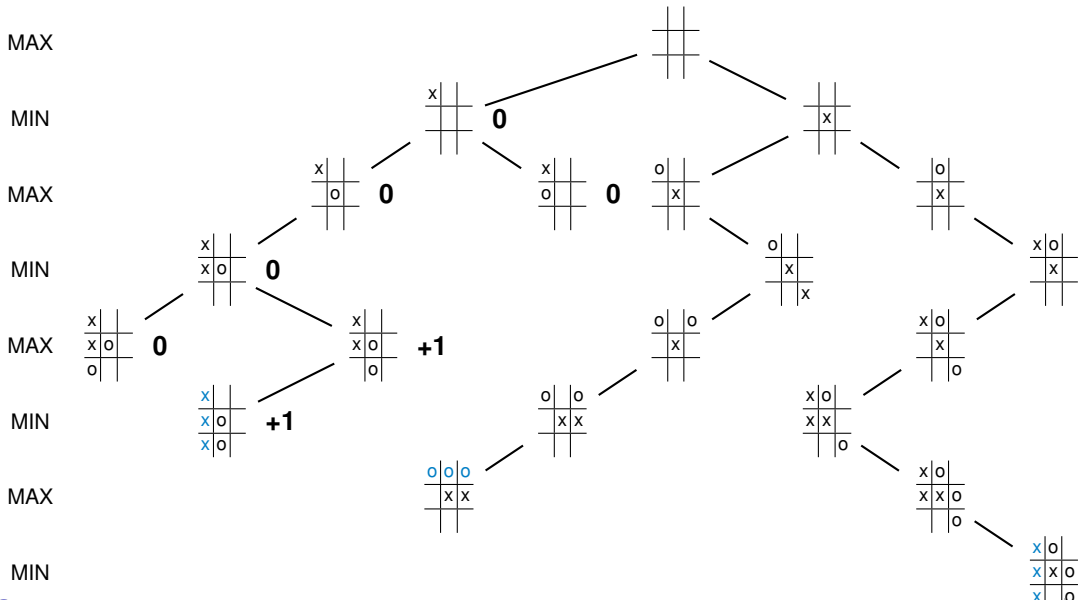

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
+1
+1

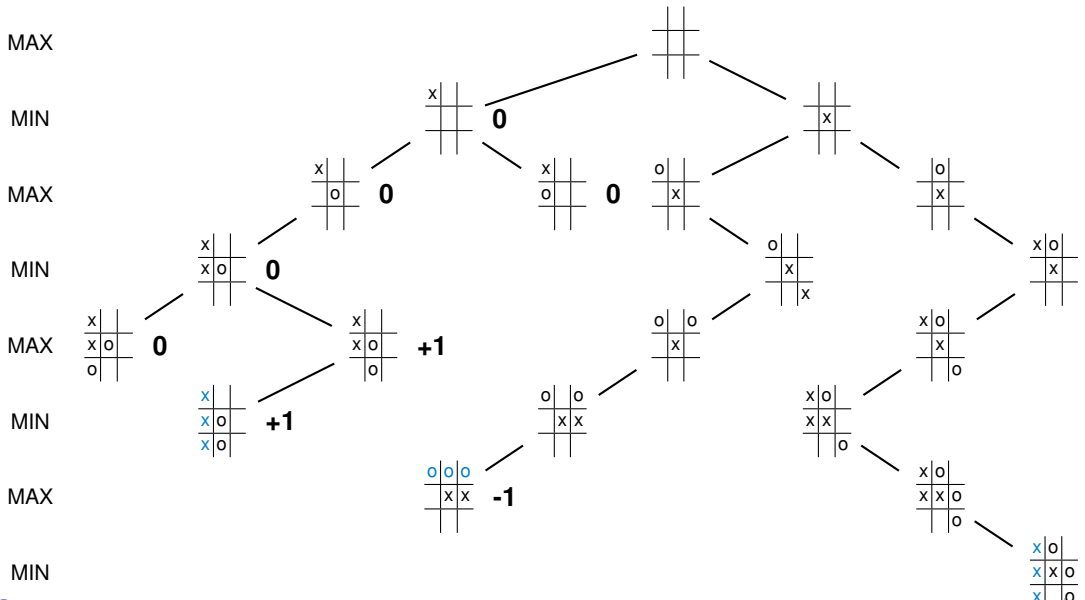

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
+1
+1

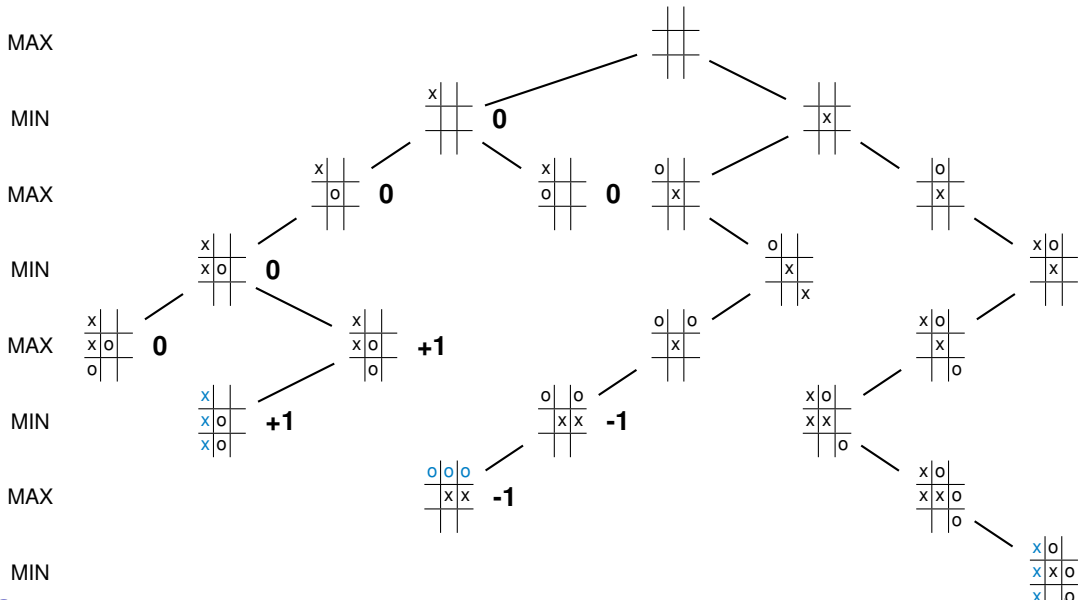

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
+1
+1

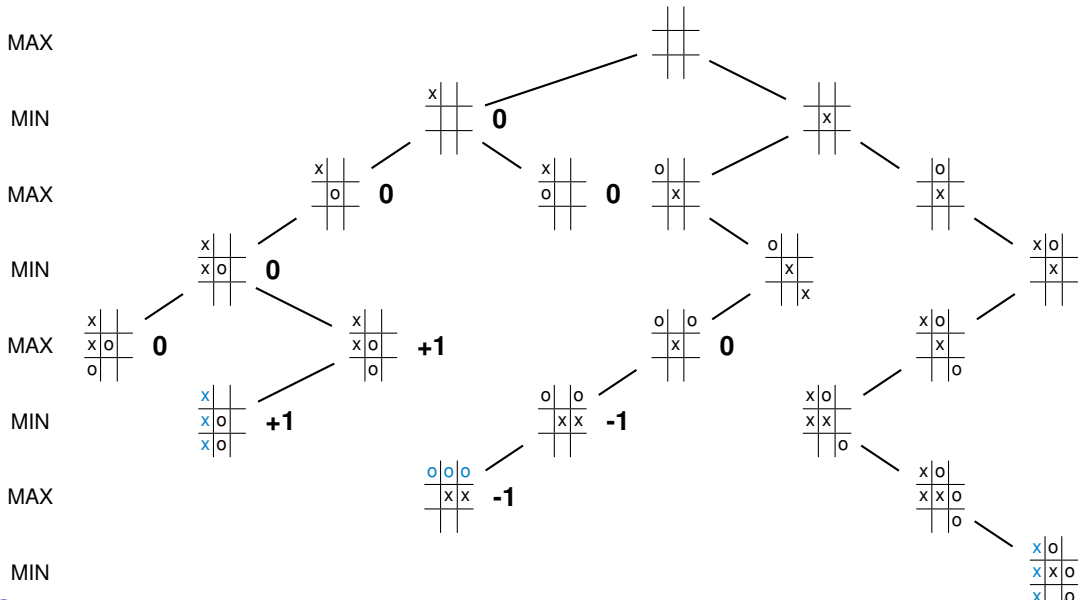

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
+1
+1

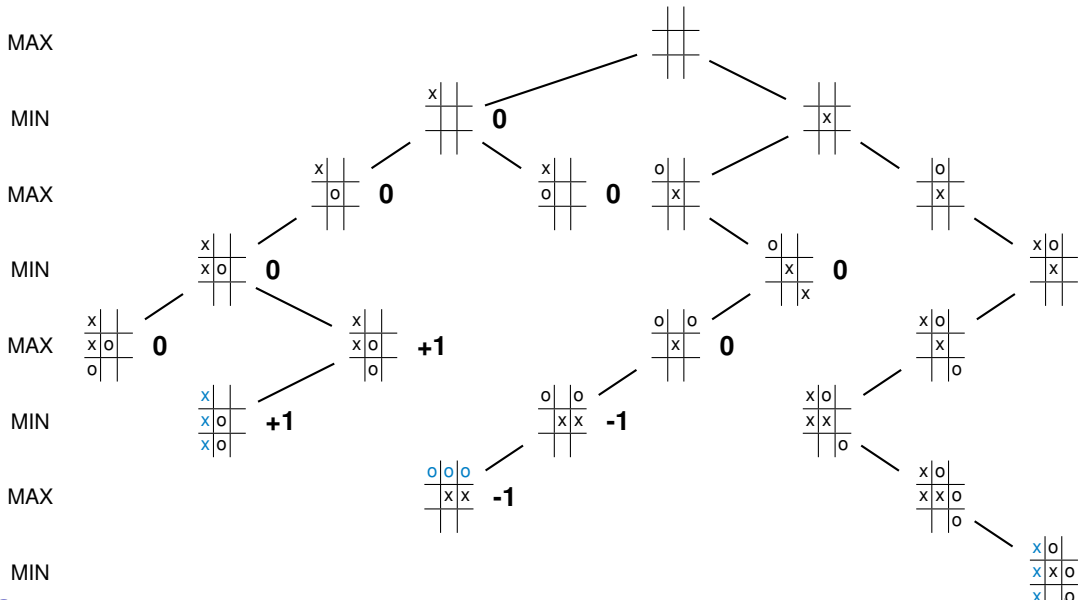

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
+1
+1

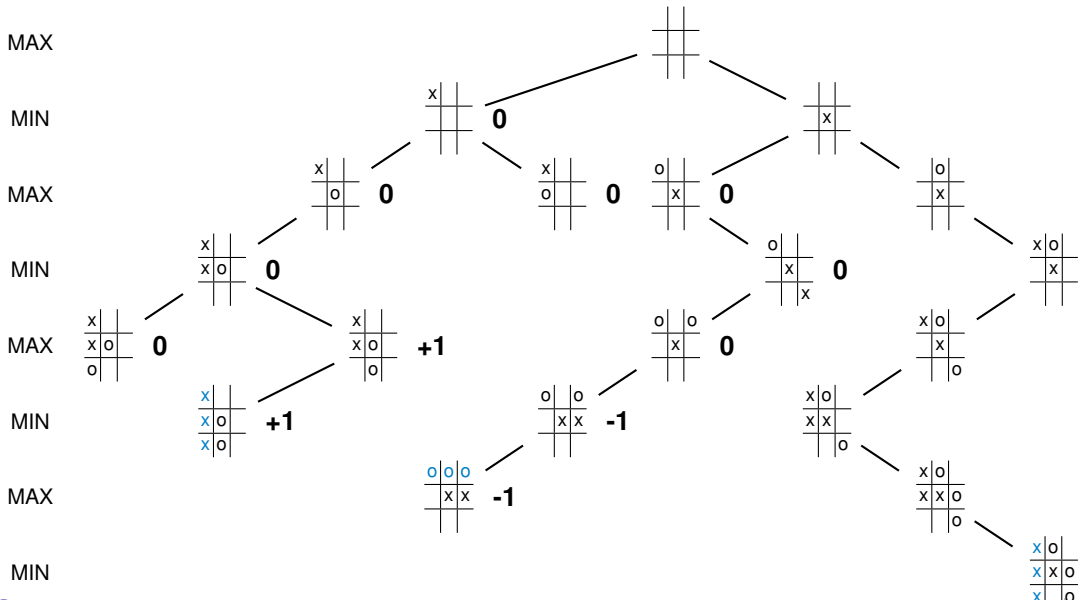

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
+1
+1


MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
+1
+1
-1

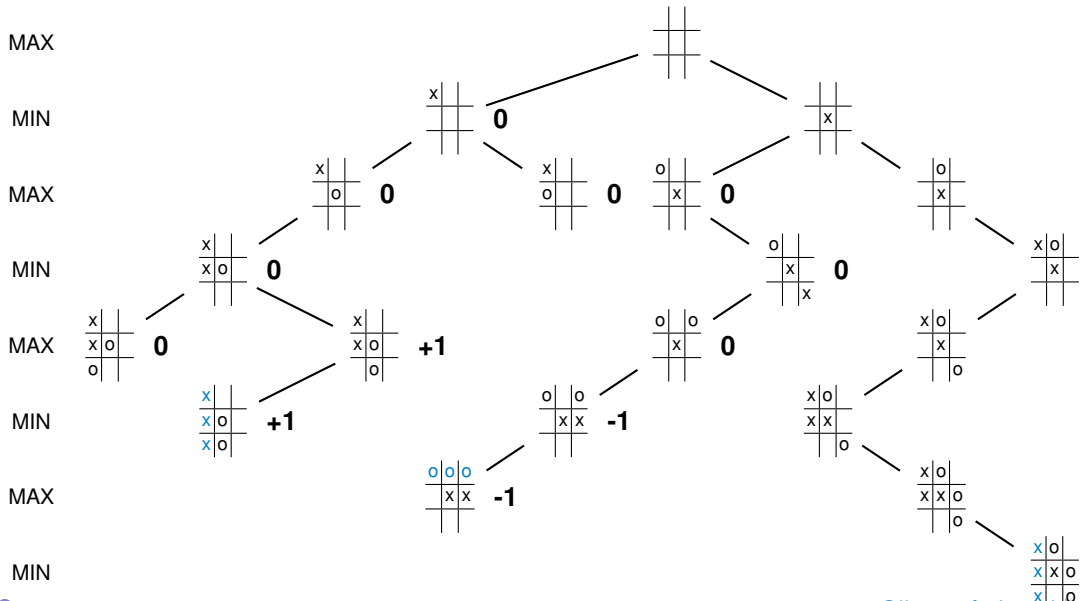

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
+1
+1
-1
-1

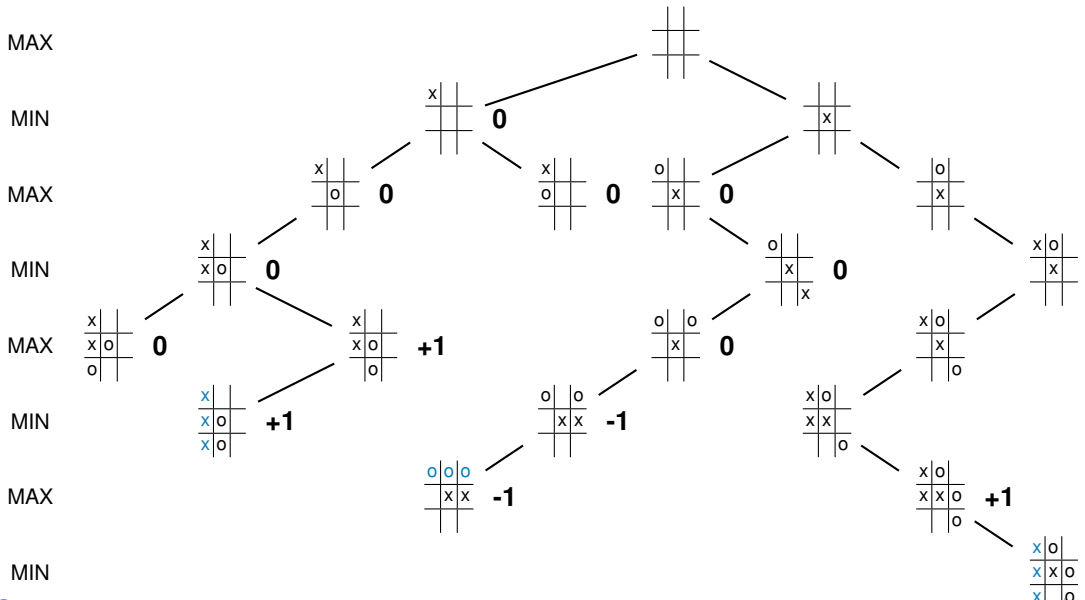

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
0
+1
+1
-1
-1

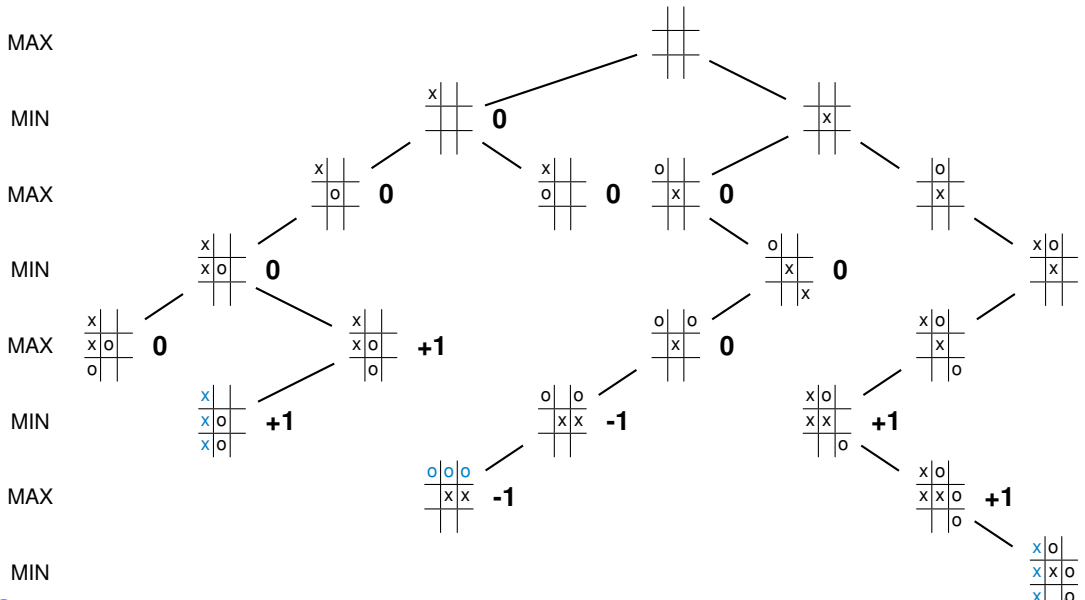

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
0
0
+1
+1
-1
-1

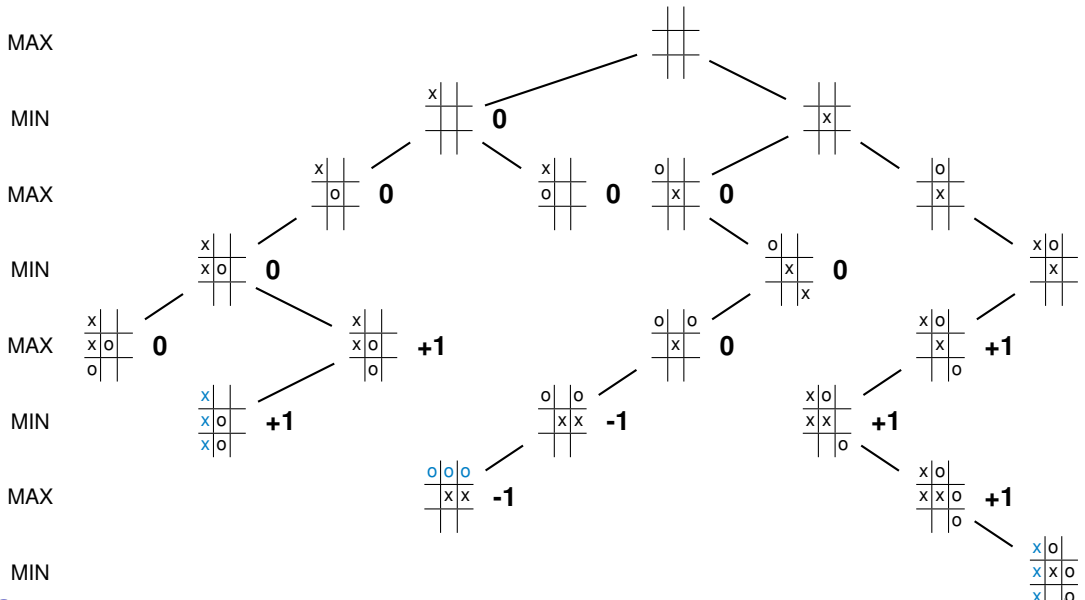

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
0
0
0
+1
+1
-1
-1

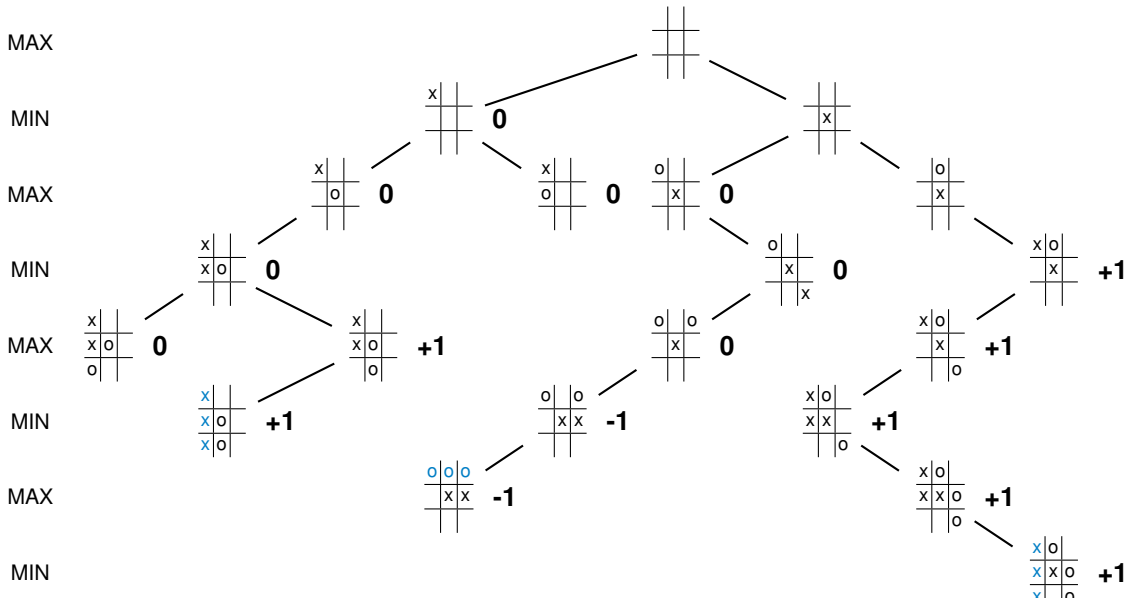
Jocuri cu 2 adversari : Minimax

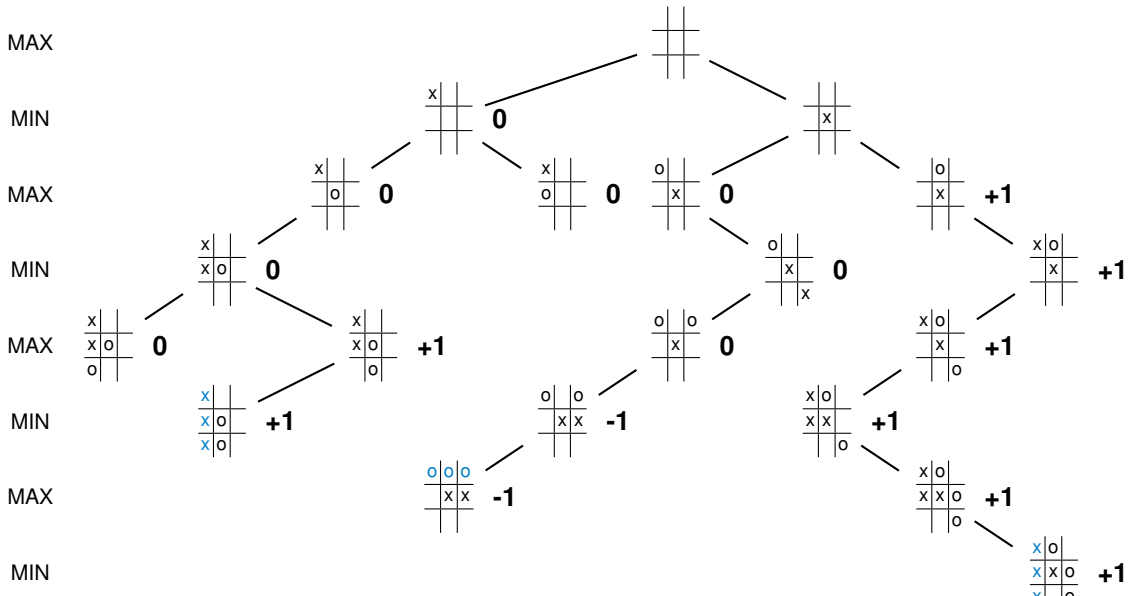


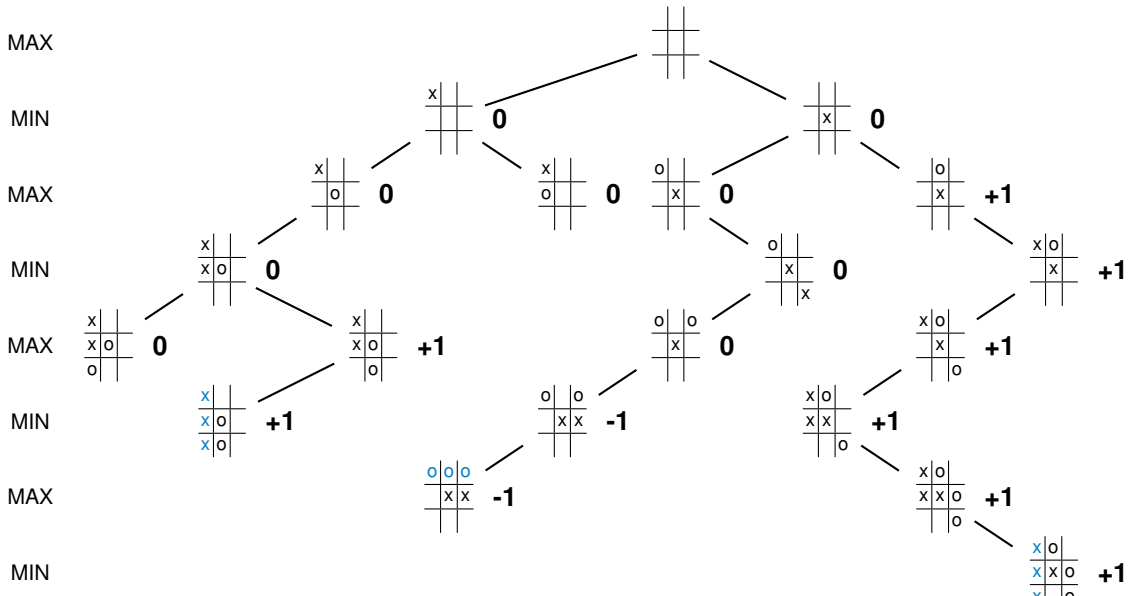

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
0
0
0
+1
+1
-1
-1
+1
+1

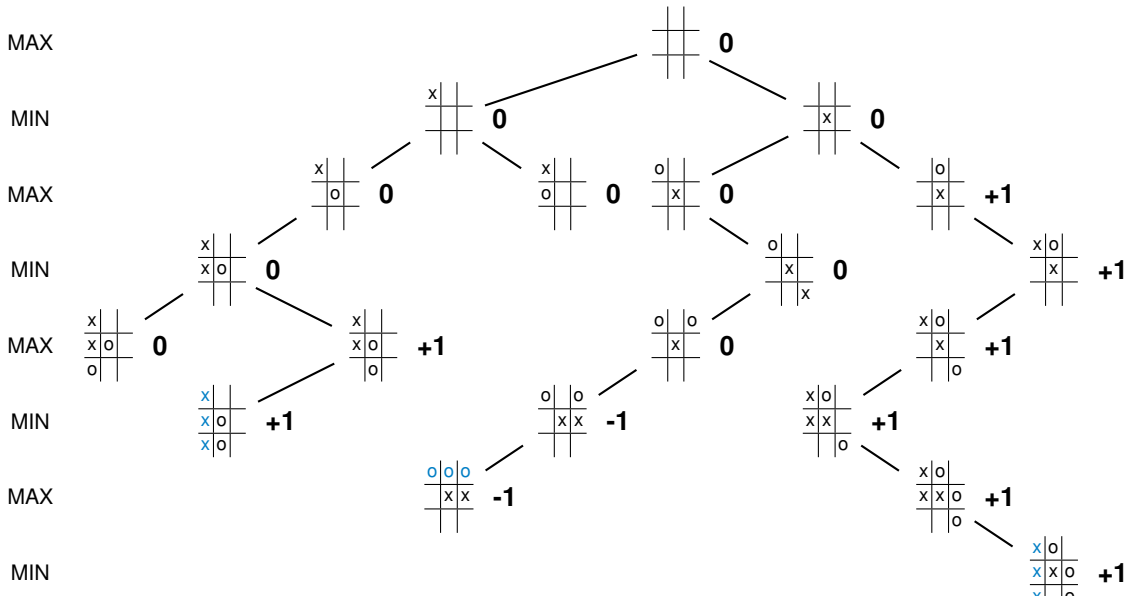

MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
0
0
0
0
+1
+1
-1
-1
+1
+1
+1


MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
0
0
0
+1
+1
-1
-1
+1
+1
+1


MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
0
0
0
+1
+1
+1
-1
-1
+1
+1
+1


MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
+1
0
0
+1
0
+1
+1
-1
-1
+1
+1
+1


MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
0
0
0
0
+1
+1
+1
+1
+1
+1
+1
-1
-1
+1


MAX
MIN
MAX
MIN
MAX
MIN
MAX
MIN
0
0
0
0
0
0
0
+1
0
0
0
+1
+1
+1
0
+1
-1
+1
-1
+1
+1

- spațiul de căutare poate fi foarte mare și parcurgerea completă nepractică

$\Rightarrow$ putem construi arborele doar până la o adâncime n

- mai departe de adâncimea n, evaluăm nodurile folosind o funcție de evaluare
  - e.g., pentru X și 0, numărul de linii pe care MAX (mai) poate câștiga, minus numărul de linii pe care MIN (mai) poate câștiga
  - dacă un jucător poate câștiga la următoarea mutare, evaluarea va da un rezultat semnificativ mai mare / mai mic

- spațiul de căutare poate fi foarte mare și parcurgerea completă nepractică

$\Rightarrow$ putem construi arborele doar până la o adâncime n

- mai departe de adâncimea n, evaluăm nodurile folosind o funcție de evaluare
  - e.g., pentru X și 0, numărul de linii pe care MAX (mai) poate câștiga, minus numărul de linii pe care MIN (mai) poate câștiga
  - dacă un jucător poate câștiga la următoarea mutare, evaluarea va da un rezultat semnificativ mai mare / mai mic
  - dar asta se poate aplica și dacă un jucător câștigă inevitabil în următoarele 2 mutări
  - ... și așa mai departe

- efectul de orizont (*horizon effect*) – este posibil ca foarte curând după adâncimea n progresul jocului să se schimbe semnificativ $\rightarrow$ aici intervine MCTS (mai jos)

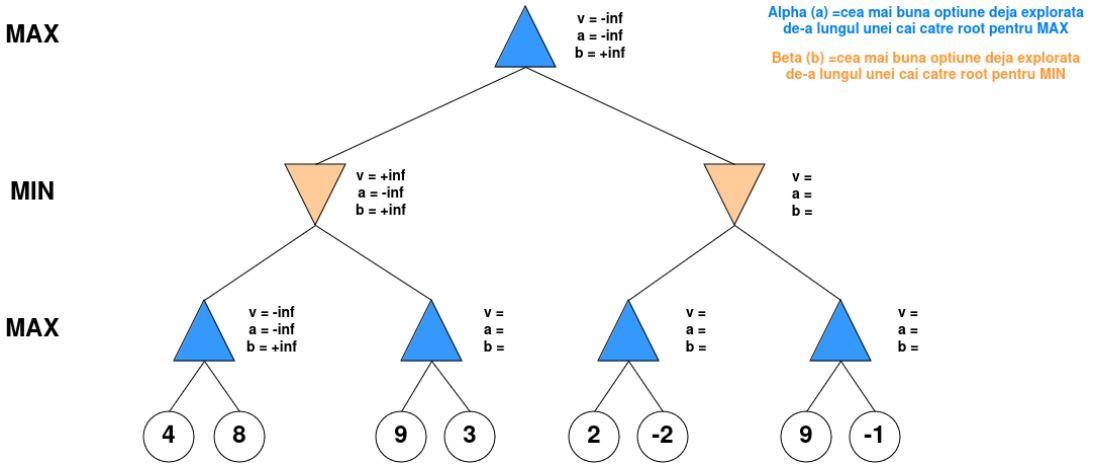
Minimax

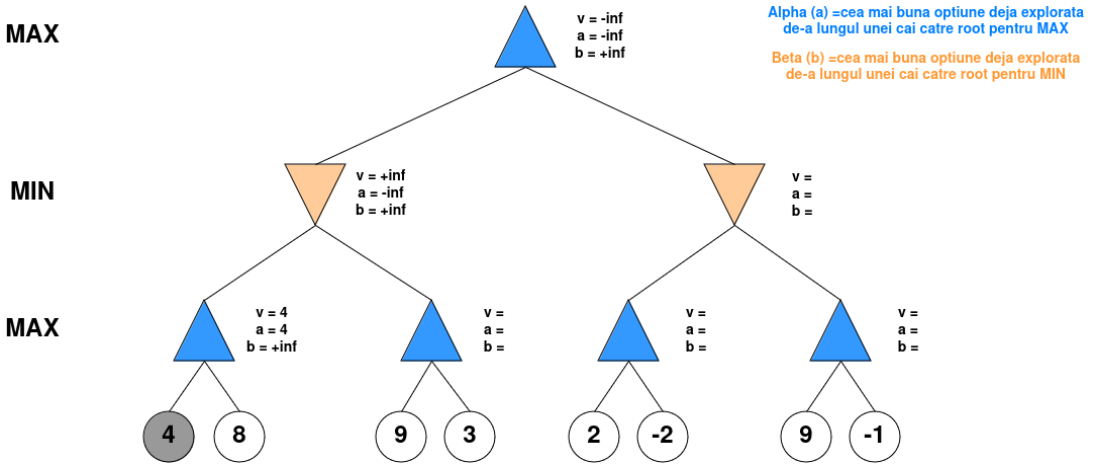
Intrări: S – starea curentă a jocului, e.g. cea inițială;
P – jucătorul care urmează, inițial MAX
n – adâncimea maximă
Ieșire: scorul maxim care poate fi obținut de jucătorul MAX

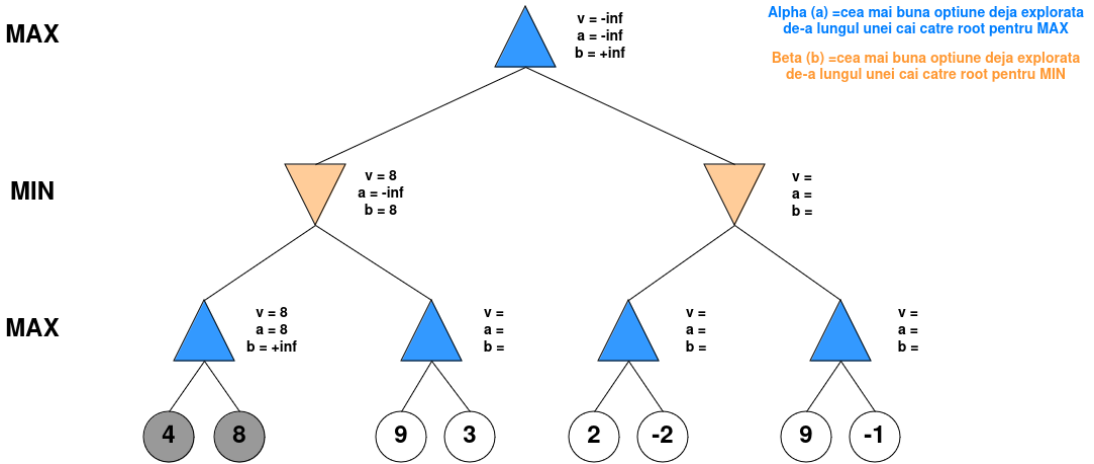
1. **dacă** S este stare finală, **atunci întoarce** $scor(S)$ *relativ la jucătorul MAX*
2. **dacă** $nivel(S) = n$ **atunci întoarce** $eval(S)$
3. **dacă** $P = \text{MAX}$ **atunci** $v = -\infty$ **altfel** $v = \infty$
4. **pentru fiecare** $S_j \in succ(S)$
5. **dacă** $P = \text{MAX}$
6. **atunci** $v \leftarrow max(v, minimax(S_j, next_player(P)))$
7. **altfel** $v \leftarrow min(v, minimax(S_j, next_player(P)))$
8. **întoarce** v

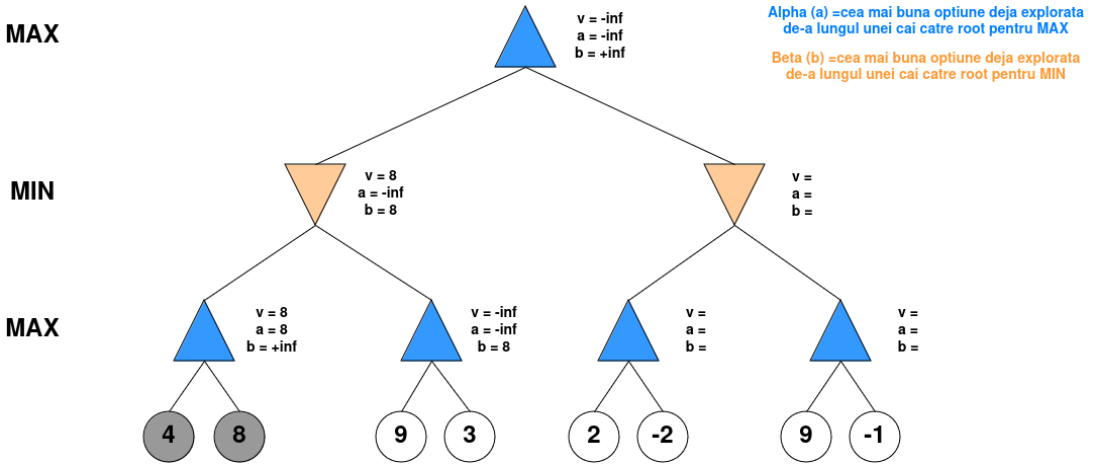
- nu este neapărat nevoie să construim întreg arborele de joc
- putem elimina (*prune*) anumite ramuri pentru că nu au cum să ofere o soluție mai bună
  → *Alpha-beta pruning* (Knuth & Moore, 1975)

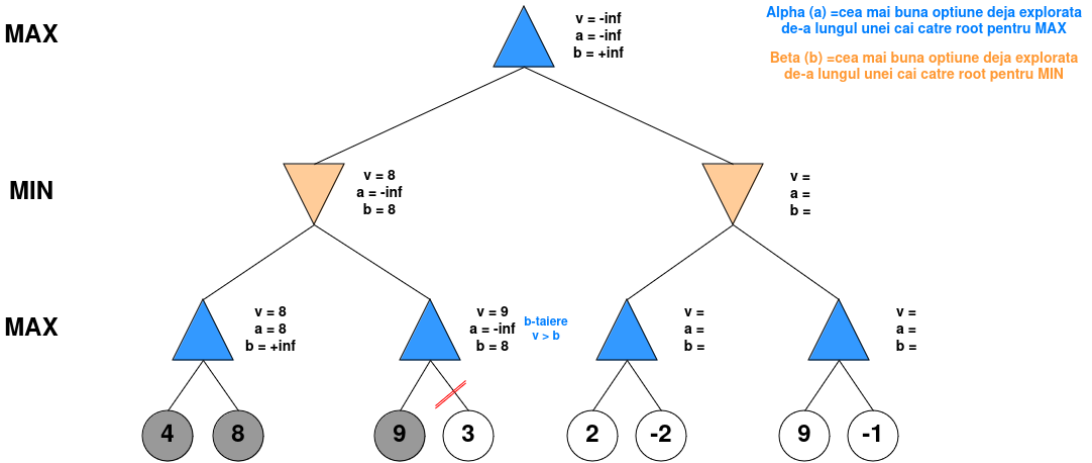
- nu este neapărat nevoie să construim întreg arborele de joc
- putem elimina (*prune*) anumite ramuri pentru că nu au cum să ofere o soluție mai bună
  $\rightarrow$ *Alpha-beta pruning* (Knuth & Moore, 1975)
- considerăm
  - α cea mai bună valoare găsită pentru jucătorul MAX
  - β cea mai bună valoare găsită pentru jucătorul MIN
- α-tăiere – nu construim un subarbore copil al unui nod MIN cu $v < \alpha$
- β-tăiere – nu construim un subarbore copil al unui nod MAX cu $v > \beta$

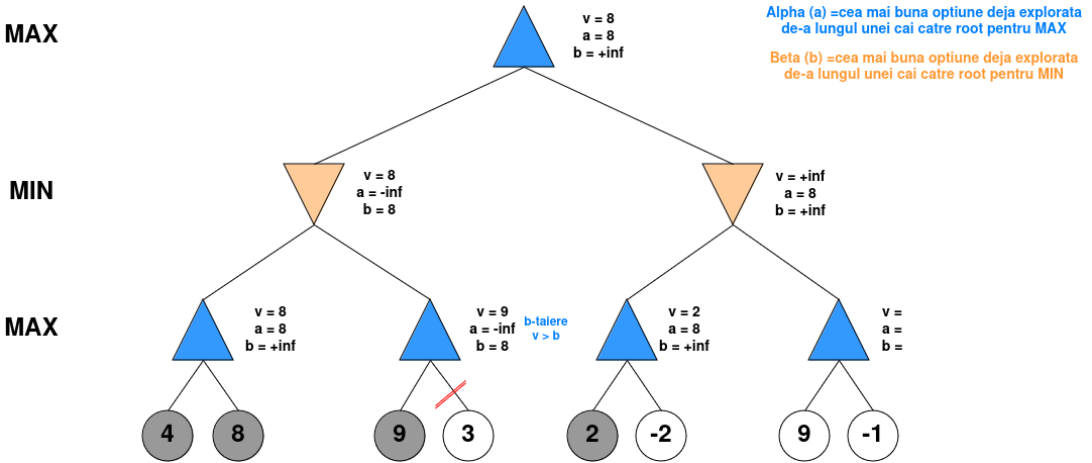

MAX
MIN
MAX
v = -inf
a = -inf
b = +inf
v = +inf
a = -inf
b = +inf
v =
a =
b =
v = -inf
a = -inf
b = +inf
v =
a =
b =
v =
a =
b =
v =
a =
b =
4
8
9
3
2
-2
9
-1
Alpha (a) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MAX
Beta (b) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MIN

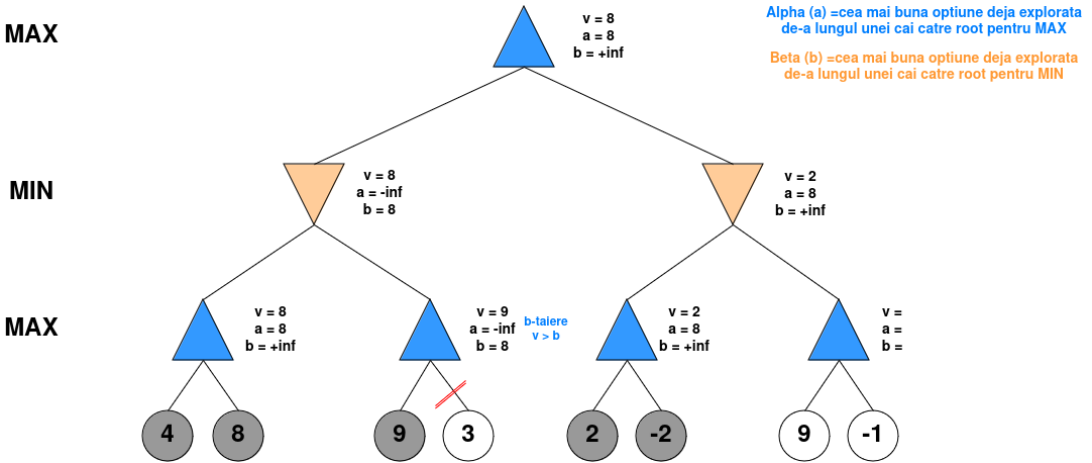

MAX
MIN
MAX
v = -inf
a = -inf
b = +inf
v = +inf
a = -inf
b = +inf
v =
a =
b =
v = 4
a = 4
b = +inf
v =
a =
b =
v =
a =
b =
v =
a =
b =
4
8
9
3
2
-2
9
-1
Alpha (a) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MAX
Beta (b) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MIN

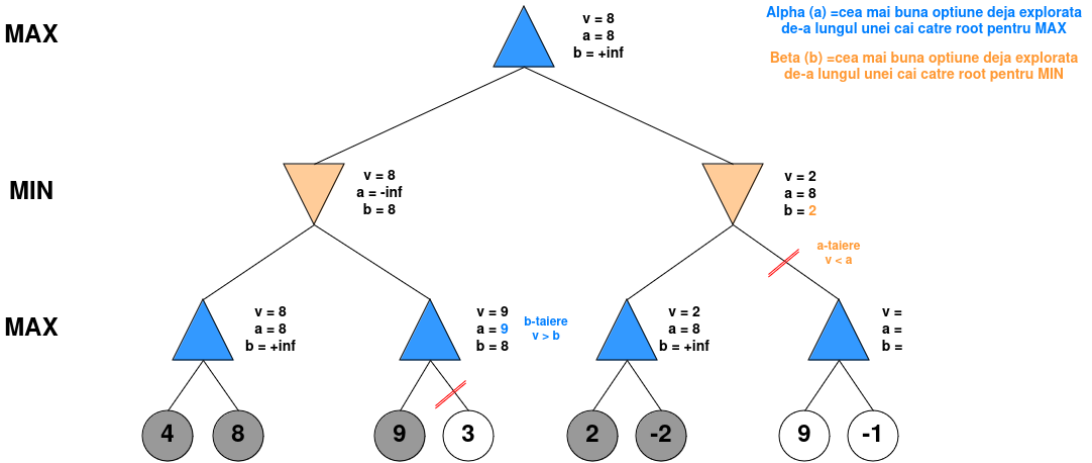

MAX
MIN
MAX
v = -inf
a = -inf
b = +inf
v = 8
a = -inf
b = 8
v =
a =
b =
v = 8
a = 8
b = +inf
v =
a =
b =
v =
a =
b =
v =
a =
b =
4
8
9
3
2
-2
9
-1
Alpha (a) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MAX
Beta (b) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MIN


MAX
MIN
MAX
v = -inf
a = -inf
b = +inf
v = 8
a = -inf
b = 8
v =
a =
b =
v = 8
a = 8
b = +inf
v = -inf
a = -inf
b = 8
v =
a =
b =
v =
a =
b =
4
8
9
3
2
-2
9
-1
Alpha (a) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MAX
Beta (b) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MIN


MAX
MIN
MAX
v = -inf
a = -inf
b = +inf
Alpha (a) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MAX
Beta (b) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MIN
v = 8
a = -inf
b = 8
v =
a =
b =
v = 8
a = 8
b = +inf
v = 9
a = -inf
b = 8
b-taiere
v > b
v =
a =
b =
v =
a =
b =
4
8
9
3
2
-2
9
-1


MAX
MIN
MAX
v = 8
a = 8
b = +inf
v = 8
a = -inf
b = 8
v = +inf
a = 8
b = +inf
v = 8
a = 8
b = +inf
v = 9
a = -inf
b = 8
b-taiere
v > b
v = 2
a = 8
b = +inf
v =
a =
b =
4
8
9
3
2
-2
9
-1
Alpha (a) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MAX
Beta (b) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MIN


MAX
MIN
MAX
v = 8
a = 8
b = +inf
v = 8
a = -inf
b = 8
v = 2
a = 8
b = +inf
v = 8
a = 8
b = +inf
v = 9
a = -inf
b = 8
b-taiere
v > b
v = 2
a = 8
b = +inf
v =
a =
b =
4
8
9
3
2
-2
9
-1
Alpha (a) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MAX
Beta (b) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MIN


MAX
MIN
MAX
v = 8
a = 8
b = +inf
v = 8
a = -inf
b = 8
v = 2
a = 8
b = 2
a-taiere
v < a
v = 8
a = 8
b = +inf
v = 9
a = 9
b = 8
b-taiere
v > b
v = 2
a = 8
b = +inf
v =
a =
b =
4
8
9
3
2
-2
9
-1
Alpha (a) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MAX
Beta (b) =cea mai buna optiune deja explorata de-a lungul unei cai catre root pentru MIN

Alfa-beta

Intrări: S – starea curentă; P – următorul jucător;
α, inițial $-\infty$; β, inițial ∞

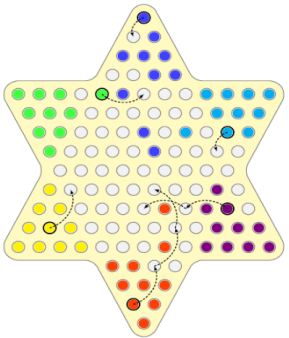
Ieșire: scorul maxim care poate fi obținut de jucătorul MAX

1. **dacă** S este stare finală, **atunci întoarce** *scor*(S) *relativ la jucătorul MAX*
2. **pentru fiecare** $S_j \in$ *succ*(S)
3. **dacă** $P = \text{MAX}$ **atunci**
4. $\alpha \leftarrow$ *max*$(\alpha,$ *alfa-beta*$(S_j,$ *next_player*$(P), \alpha, \beta))$
5. **dacă** $\alpha > \beta$ **atunci întoarce** β *β cutoff*
6. **altfel**
7. $\beta \leftarrow$ *min*$(\beta,$ *alfa-beta*$(S_j,$ *next_player*$(P), \alpha, \beta))$
8. **dacă** $\alpha > \beta$ **atunci întoarce** α *α cutoff*
9. **dacă** $P = \text{MAX}$ **atunci întoarce** α **altfel** β

- putem ordona (euristic) succesorii în așa fel încât să construim întâi cei mai buni succesori

Chinese checkers

- variație germană a jocului american Halma, inventat de George H. Monks în 1880.
- joc cu informație perfectă pentru 2-6 jucători
- fiecare jucător deplasează 10 piese dintr-o poziție de start într-o poziție finală
- pisele se mută în poziția vecină sau sărind peste una sau o serie de piese alăturate

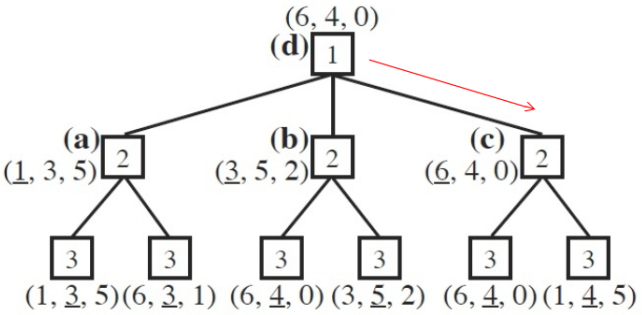


Jocuri cu mai mulți jucători

- Nu există algoritmi buni consacrați.
- avem 2 strategii:
  - **MAX**^{n} – generalizare a Minimax pentru **n** jucători
  - **Paranoic** – reduce la joc cu 2 jucători în care se presupune că toți ceilalți colaborează împotriva jucătorului MAX

Jocuri cu mai mulți jucători : Max^n

- Generalizarea Minimax pentru n jucători
- Nodurile arborelui de joc sunt n-tuple, în care elementul pe poziția i este scorul jucătorului i.
- Pentru non-frunze, valoarea *Max*^{n} a unui nod în care jucătorul i mută este valoarea *Max*^{n} a succesorului pentru care a i-a componentă din vector este maximă.



Max^n
Intrări: N – starea curentă;
P – jucătorul care urmează să mute;
n – adâncime maximă
Ieșire: scorul maxim care poate fi obținut de jucătorul MAX

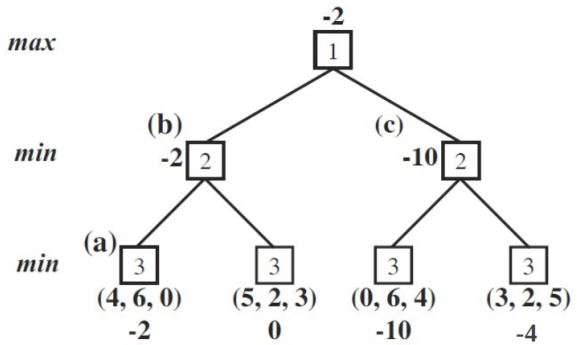
1. **dacă** N este stare finală **atunci întoarce** $N[P]$
2. **dacă** *nivel*$(N) = n$ **atunci întoarce** *eval*(N, P)
3. *next* $\leftarrow \{\}$
4. **pentru fiecare** $s_j \in$ *succ*(N)
5. *next* $\leftarrow$ *next* $\cup$ *Max*$^n(s_j,$ *next_player*$(P))$
6. *best* $\leftarrow$ *tuple* $\in$ *next*, cu *tuple*$[P]$ maxim
7. **întoarce** *best*

Atenție!

- Pot exista mai multe valori egale Max^n într-un arbore
- Rezultatul poate depinde drastic de felul în care se face alegerea – e.g. $(\underline{2}, 3, 3)$ vs. $(\underline{2}, 1, 7)$
- Alfa-beta în adâncime (*deep pruning*) nu poate fi aplicat
- Max^n-shallow pruning, Korf, 1991 (analog cu alfa-beta dar cu performante proaste)

Jocuri cu mai mulți jucători : Paranoic

- **Paranoic** – reduce jocul la 2 jucători – MAX și toți ceilalți jucători ca un adversar colectiv
- este o strategie paranoică – consideră că toți ceilalți jucători se aliază împotriva lui MAX
- în fiecare nod scorul este scorul jucătorului MAX minus scorul tutoror celorlalți jucători
- fiind la fel ca Minimax:
  - există o unică valoare în fiecare nod
  - se poate utiliza tăiere alfa-beta, dar pe măsură ce numărul jucătorilor crește beneficiul adus de tăiere scade.
  - pentru jocuri cu 3-6 jucători, adâncimea este cu 20-50% mai mare decât la Max^n.


max
min
min
-2
1
(b)
-2
2
(c)
-10
2
(a)
3
3
3
3
(4, 6, 0)
-2
(5, 2, 3)
0
(0, 6, 4)
-10
(3, 2, 5)
-4

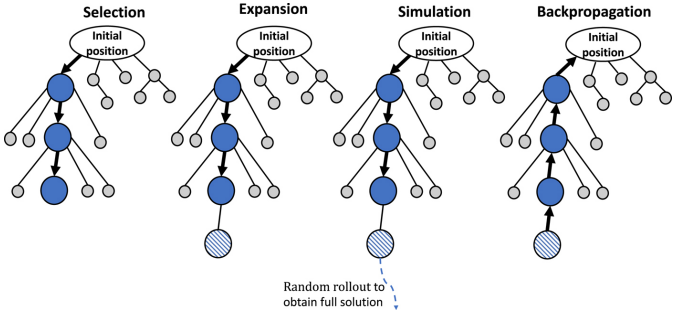
Monte Carlo Tree Search

- **MCTS** – algoritm probabilistic care utilizează o serie de simulări aleatoare pentru a expanda selectiv arborele de joc.
- metodă bună pentru luarea deciziilor în probleme cu un spațiu de căutare mare.
- best-first search care înlocuiește euristica cu rezultatele unor simulări Monte-Carlo.
  - metodele de tip Monte Carlo folosesc eșantionare aleatoare repetată pentru a obține informații despre procese determinsite.
- se bazează pe 2 ipoteze:
  1. adevărata *valoare a unei acțiuni* (mutare în joc) poate fi aproximată utilizând simulări aleatoare.
  2. valorile astfel obținute pot fi utilizate pentru a *ajusta politica de selecție* spre o cea mai bună strategie.

Monte Carlo Tree Search

- baza metodei este utilizarea unei unde de joc / simulări (*playout*).
- Playout – un joc jucat cu mutări aleatoare dintr-o anumită stare *până la starea finală*, obținându-se un scor.
  - nu utilizează spațiu suplimentar pentru că nu construiește noduri în arbore – în urma simulării se reține doar scorul final.
- fiecărui nod construit i se asociază un merit (sau calitate – *quality*), bazat pe rezultatul simulărilor.
- se construiește un arbore de joc parțial, format din stările cele mai promițătoare.
- algoritm de tip *stop-anytime* – ne putem opri oricând și avem o soluție, care este cu atât mai bună cu cât am explorat mai mult.

Monte Carlo Tree Search : Etapele algoritmului

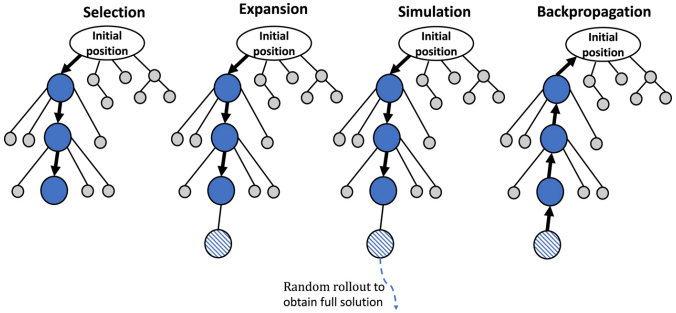


La fiecare iterație a algoritmului avem 4 pași:

selecție | construcție(expandare) | simulare | propagare (backpropagation)

Căutăm un nod care nu este încă (complet) expandat – pornim de la rădăcină și aplicăm recursiv o politică de selecție a copiilor pentru a găsi cel mai potrivit nod căruia să îi construim un copil.

Monte Carlo Tree Search : Etapele algoritmului

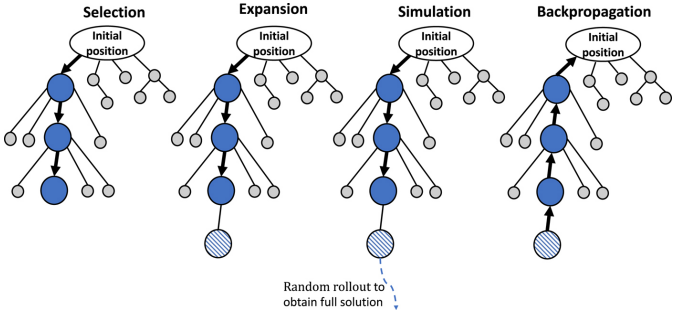


La fiecare iterație a algoritmului avem 4 pași:

selecție | construcție(expandare) | simulare | propagare (backpropagation)

Construim unul sau mai multe noduri noi, corespunzătoare unor acțiuni încă neexplorate.

Monte Carlo Tree Search : Etapele algoritmului

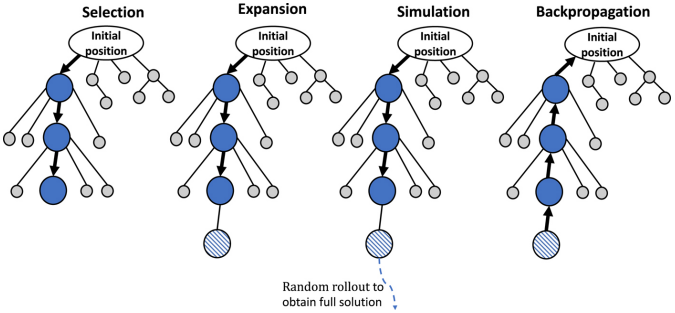


La fiecare iterație a algoritmului avem 4 pași:

selecție | construcție(expandare) | simulare | propagare (backpropagation)

Realizăm o *simulare* pornind din nodul/nodurile noi.

Monte Carlo Tree Search : Etapele algoritmului



La fiecare iterație a algoritmului avem 4 pași:

selecție | construcție(expandare) | simulare | propagare (backpropagation)

Propagăm către rădăcină rezultatul obținut.

Monte Carlo Tree Search : Algoritm

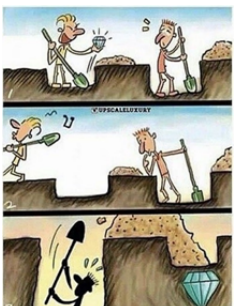
MCTS(rădăcină)

1. $(v, s) \leftarrow$ nodul rădăcină, corespunzător stării s_0, $parinte(v) \leftarrow \perp$
2. **pentru fiecare** unitate de buget **repetă**
3. **cât timp** s nu este stare finală
4. **dacă** v este complet expandat **atunci** $(v, s) \leftarrow select(v)$ **altfel break**
5. $a \leftarrow alege_actiune(s)$ *am ajuns la nodul care trebuie expandat*
6. construiește (v', s'), cu $s' \leftarrow next(s, a)$ și $parinte(v') \leftarrow v$; $(v, s) \leftarrow (v', s')$
7. **cât timp** s nu este stare finală
8. $a \leftarrow$ acțiune disponibilă în s, aleasă aleator
9. $s \leftarrow next(s, a)$
10. $r \leftarrow recompensa(s')$
11. **cât timp** $v \neq \perp$
12. $N(v) \leftarrow N(v) + 1$
13. $Q(v) \leftarrow actualizare_Q(v, r)$
14. $v \leftarrow parinte(v)$
15. $(v, s) \leftarrow select(radacina)$
16. **întoarce** a, cu $s = next(s_0, a)$ *cea mai bună acțiune*

Monte Carlo Tree Search : Discuție

Ce strategii se folosesc pentru selecție?

- Exploatare: Selectăm stări din care s-a câștigat des și au fost parcurse de multe ori
- Explorare: Selectam stări cu puține simulări anterioare



Monte Carlo Tree Search : Discuție

- Selecție folosind UCT – *Upper Confidence Bound 1 applied to trees*

$$select(s) \leftarrow argmax_{s' \in succ(s)} \left(\frac{Q(s')}{N(s')} + C \cdot \sqrt{\frac{2 \cdot \ln(N(s))}{N(s')}} \right)$$

- Echilibrează calitatea unui nod copil cu gradul de explorare al părintelui în raport cu copilul.

Monte Carlo Tree Search : Discuție

- Backpropagation poate folosi, pentru propagarea de la s' la părintele s:

$$V(s) \leftarrow \frac{V_{med} \cdot W_{med} + V_r \cdot N_r}{W_{med} \cdot N_r}$$

  - V_{med} — media valorilor nodurilor copii
  - W_{med} — ponderea acestei medii
  - V_r — mutarea cu cel mai mare număr de simulări
  - N_r — numărul de ori de care s-a jucat V_r

Monte Carlo Tree Search : Discuție

- MCTS converge *lent* spre valorile Minimax, dar este mai eficient decât tăierea alfa-beta.
- nu găsește întotdeauna cea mai bună mutare, dar are, în general, un succes rezonabil în cazul alegerii mutărilor care duc la șanse mari de câștig.
- poate să nu detecteze ramuri care, folosind *o anumită line de joc*, duc la un rezultat semnificativ diferit
- *Light playouts* — playout complet aleator
- *Heavy playouts* — euristici/politici pentru selectarea mișcării următoare în timpul simulării

Monte Carlo Tree Search : Discuție

- MoGo – A participat în 30 de turnee între 2006 și 2010 (9 x 9 GO)
  - A câștigat contra profesioniștilor
  - MoGo învinge pe campionul Myungwan Kim, august 2008, utilizand MCTS
- FUEGO – a învins mai mulți campioni la 9 x 9 GO în 2009
- MoHex – devine campion la jocul Hex în 2009

Mulțumesc!



https://forms.gle/DJUdkejstkvyNRF5A

Feedbackul este binevenit!